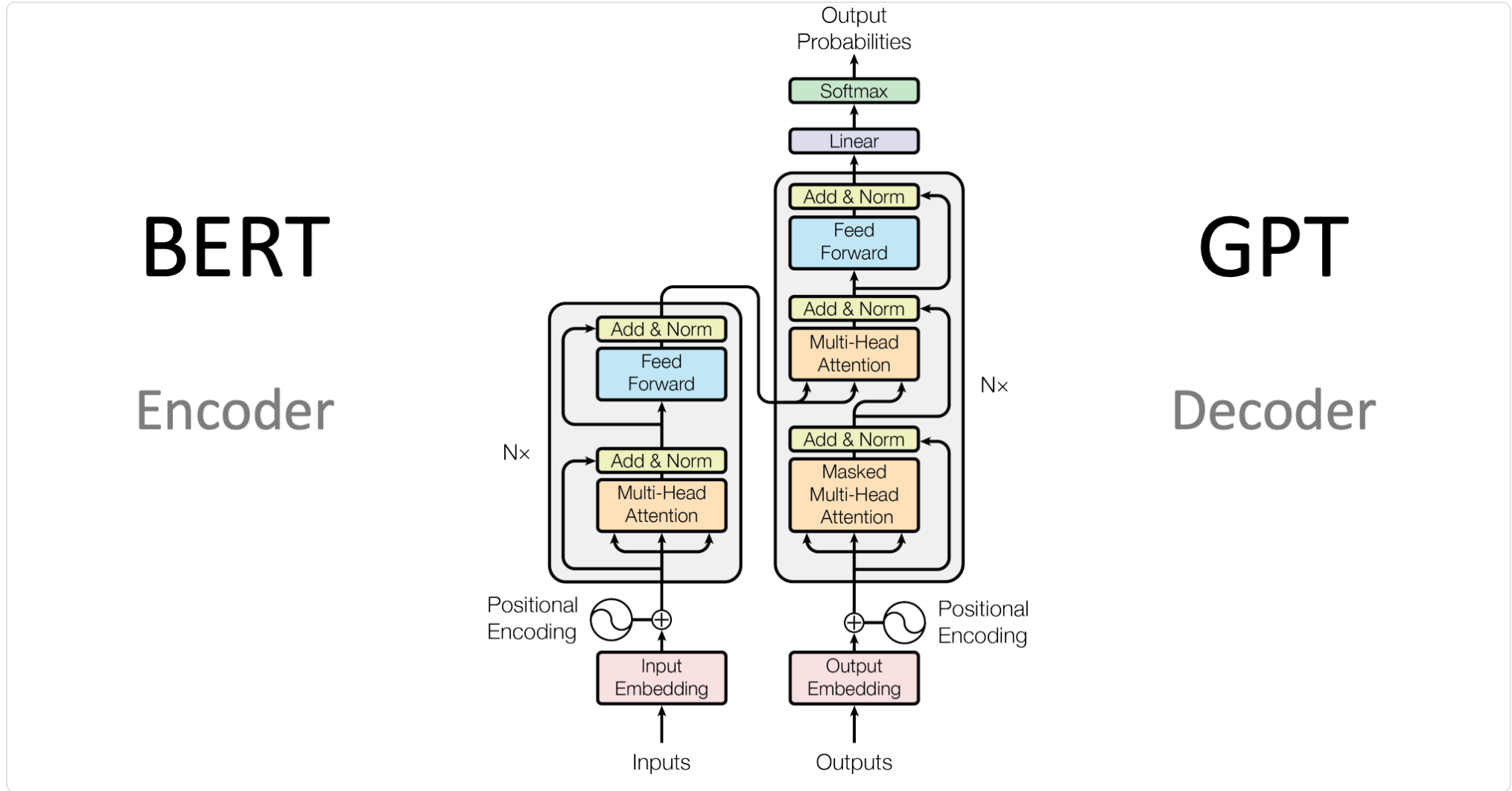


Foundation Models, Transformers, BERT and GPT

Posted Feb 23, 2023 • Updated Nov 25, 2024



By [Niklas Heidloff](#)

7 min read

Since I'm excited by the incredible capabilities which technologies like ChatGPT and Bard provide, I'm trying to understand better how they work. This post summarizes my current understanding about foundation models, transformers, BERT and GPT.

Note that I'm only learning these concepts and not everything might be fully correct but might help some people to understand the high level concepts.

I know that there are many more and more modern Foundation Models than BERT and GPT, but I want to start 'simple' and these two models are probably the most known ones these days.

The technologies below are not trivial and there are lots of articles, papers and full courses even on certain aspects of each technology only. Instead of going into detail, I try to explain what they do and what concepts they use.

Foundation Models

BERT and GPT are both foundation models. Let's look at the definition and characteristics:

- Pre-trained on different types of unlabeled datasets (e.g., language and images)
- Self-supervised learning
- Generalized data representations which can be used in multiple downstream tasks (e.g., classification and generation)
- The Transformer architecture is mostly used, but not mandatory

Read my blog [Foundation Models at IBM](#) to find out more.

Most foundation models use the transformer architecture. Let's look at the [definition](#):

A transformer is a deep learning model that adopts the mechanism of self-attention, differentially weighting the significance of each part of the input data. It is used primarily in the fields of natural language processing and computer vision.

In 2017 transformers were introduced: [Attention is all you need](#). They are the next generation of Recurrent Neural Networks and Long Short-Term Memory architectures and have several benefits:

- Parallel processing: Increases performance and scalability
- Bidirectionality: Allows understanding of ambiguous words and coreferences

The original transformer architecture defines two main parts, an encoder and a decoder. However, not all foundation models use both parts. BERT only uses encoders, GPT only decoders. More on this later.

Attention

Both encoders and decoders use the concept of 'attention'. Attention basically means to focus on the important pieces of information and to blend out the unimportant pieces. I like to compare this with 'fast reading'. Rather than reading full articles or even full books, I often browse chapter titles, first words of paragraphs and scan paragraphs for keywords to find what I'm looking for.

The words of an article, the parts of an image or the words in a sentence that should get most attention change dependent on what you are looking for. Let's look at a simple example sentence:

"Sarah went to a restaurant to meet her friend that night."

The following words should get attention for the following queries:

- What? -> 'went', 'meet'
- Where? -> 'a restaurant'
- Who? -> 'Sarah', 'her friend'
- When? -> 'that night'

To determine the attention of words (more exactly tokens) 'queries', 'keys' and 'values' are used by encoders and decoders in transformers. All of them are presented in vectors. Keys are found for certain queries if they are closest to the query vector. Keys are an encoded representation for values, in simple cases they can be the same.

There are different algorithms to implement the attention concept. I think an easy way to understand how this can work is to rank words high that are often used together in sentences. For example, 'where' and 'restaurant' have probably a closer relation than 'restaurant' and 'faith'. So, for the query 'where' the word 'restaurant' gets more attention.

Encoders and Decoders

As mentioned, there are encoders and decoders. BERT uses encoders only, GTP uses decoders only. Both options understand language including syntax and semantics. Especially the next generation of large language models like GPT with billions of parameters do this very well.

The two models focus on different scenarios. However, since the field of foundation models is evolving, the differentiation is often fuzzier.

- BERT (encoder): classification (e.g., sentiment), questions and answers, summarization, named entity recognition
- GPT (decoder): translation, generation (e.g., stories)

The outputs of the core models are different:

- GPT (decoder): Next words with probabilities

Both models are pretrained and can be reused without intensive training. Some of them are available as open source and can be downloaded from communities like Hugging Face, others are commercial. Reuse is important, since trainings are often very resource intensive and expensive which few companies can afford.

The pretrained models can be extended and customized for different domains and specific tasks. Layers can sometimes be reused without modifications and more layers are added on top. If layers need to be modified, the new training is more expensive. The technique to customize these models is called `Transfer Learning` , since the same generic model can easily be transferred to other domains.

BERT - Encoders

BERT uses the encoder part of the transformer architecture so that it understands semantic and syntactic language information. The output of BERT are embeddings, not predicted next words. To leverage these embeddings, other layer(s) need to be added on top, for example text classification or questions and answers.

BERT uses a genius trick for the training. For supervised training it is often expensive to get labeled data, sometimes it's impossible. The trick is to use masks as I described in my post [Evolution of AI explained via a simple Sample](#). Let's take a simple example, an unlabeled sentence:

“Sarah went to a restaurant to meet her friend that night.”

This is converted into:

- Text: “Sarah went to a restaurant to meet her MASK that night.”
- Label: “Sarah went to a restaurant to meet her friend that night.”

Note that this is a very simplified description only since there aren't 'real' labels in BERT.

In other words, BERT produces labeled data for originally unlabeled data. This technique is called `Self-Supervised Learning` . It works very well for huge amounts of data.

In masked language models like BERT, each masked word (token) prediction is conditioned on the rest of the tokens in the sentence. These are received in the encoder which is why you don't need a decoder.

GPT - Decoders

In language scenarios decoders are used to generate next words, for example when translating text or generating stories. The outputs are words with probabilities.

Decoders also use the attention concepts and even two times. First when training models, they use `Masked Multi-Head Attention` which means that only the first words of the target sentence are provided so that the model can learn without cheating. This mechanism is like the `MASK` concept from BERT.

After this the decoder uses `Multi-Head Attention` as it's also used in the encoder. Transformer based models that utilize encoders and decoders use a trick to be more efficient. The output of the encoders is feed as input to the decoders, more precisely the keys and values. Decoders can invoke queries to find the closest keys. This allows, for example, to understand the meaning of the original sentence and translate it into other languages even if the number of resulting words and the order changes.

GPT doesn't use this trick though and only use a decoder. This is possible since these types of models have been trained with massive amounts of data (`Large Language Model`). The knowledge of encoders is encoded in billions of parameters (also called weights). The same knowledge exists in decoders when trained with enough data.

Note that ChatGPT has evolved these techniques. To prevent hate, profanity and abuse, humans need to label some data first. Additionally `Reinforcement Learning` is applied to improve the quality of the model (see [ChatGPT: Optimizing Language Models for Dialogue](#)).



There are many good articles, videos and courses. Here are some of the ones I read or watched:

- Course: [Natural Language Processing Demystified](#)
- YouTube channel: [CodeEmporium](#)
- Article: [What Is ChatGPT Doing ... and Why Does It Work?](#)
- Article: [10 Things You Need to Know About BERT and the Transformer Architecture That Are Reshaping the AI Landscape](#)
- Article: [Transformer’s Encoder-Decoder: Let’s Understand The Model Architecture](#)
- [NLP - BERT & Transformer](#)



OLDER

[IBM Watson NLP Use Cases](#)

NEWER

[ChatGPT-like Functionality in IBM Watson Assistant](#)